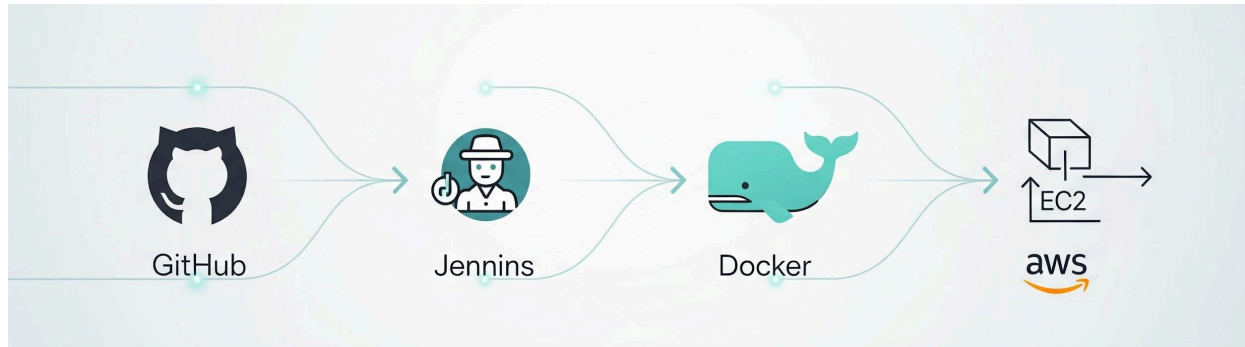




Auto Deploy Website using Jenkins & Docker: A DevOps Project

This document outlines a simple, step-by-step DevOps project designed for beginners to learn core concepts by automating the deployment of a static website.

Project Name

"Auto Deploy Website using Jenkins & Docker"

What Students Will Learn

Students will gain hands-on experience with the following key technologies and concepts:

- **GitHub:** To store and manage the application source code.
- **Jenkins:** To automate the build and deployment process.
- **Docker:** To package the application and its dependencies into a portable container.
- **AWS EC2:** To serve as the host for the Jenkins and Docker environments.

ARCHITECTURE

The final architecture for this project is a streamlined deployment pipeline:

EC2 → GitHub → Jenkins → Docker → EC2 → Browser

STEP 1: Create Simple Project

This step involves creating a basic HTML file and a **Dockerfile** to set up the web application.

1. Create the Website File:

```
None
echo "Hello from DevOps Project" > index.html
```

2. Create the Dockerfile:

```
None
vi Dockerfile
```

Add the following content:

```
None
FROM nginx
COPY index.html /usr/share/nginx/html/
```

3. Push to GitHub:

Initialize a Git repository and push the files to GitHub.

```
None
git init
git add .
git commit -m "first commit"
git remote add origin <repo-url>
git push -u origin main
```

*Note: Replace **<repo-url>** with the URL of your GitHub repository.*

STEP 2: Setup Jenkins Job

Create a new job in Jenkins to manage the project.

- Navigate to Jenkins and click **New Item**.
 - Name the project: `simple-devops-project`
 - Select the **Freestyle Project** option.
-

STEP 3: Connect GitHub

Configure the Jenkins job to pull code from the GitHub repository.

- In the job configuration, go to **Source Code Management** and select **Git**.
 - Paste your repository URL: `https://github.com/<your-repo>.git`
Note: Replace <your-repo> with your actual GitHub username and repository name.
-

STEP 4: Add Build Step

Add an **Execute Shell** step to the Jenkins job that contains the commands for building the Docker image and deploying the container.

- In the job configuration, go to **Build** and click **Add build step** → **Execute Shell**.
- Add the following shell commands:

```
None
echo "Starting Build"

# Build Docker Image
docker build -t myapp .

# Stop old container

docker stop mycontainer || tru
```

```
e
docker rm mycontainer || true
```

None

```
# Run new container
docker run -d -p 8081:80 --name mycontainer myapp

echo "Deployment Completed"
...
```

STEP 5: Give Permission

Ensure the Jenkins user has the necessary permissions to interact with Docker. Run the following commands on your EC2 instance:

None

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

► STEP 6: Build Project

Trigger the automated build and deployment process.

- In the Jenkins job, click **Build Now**.

STEP 7: Open Application

Access the newly deployed application in a web browser.

- The website should be accessible at: <http://<EC2-IP>:8081>

 **Your website is live!**

HOW IT WORKS

The deployment flow can be summarized in these steps:

1. Code is stored in GitHub.
2. Jenkins pulls the latest code from GitHub.
3. Docker uses the `Dockerfile` to build a new `myapp` image.
4. The old container (`mycontainer`) is stopped and removed, and the new container runs, exposing the internal port 80 to the host port 8083.
5. The website is deployed and accessible via the EC2 instance's IP address.

FINAL FLOW

Component	Role
Developer	Commits code changes
GitHub	Stores the source code
Jenkins	Automates the build and deploy task
Docker	Creates and runs the application container
EC2	Hosts Jenkins and the Docker container
Browser	Displays the deployed website

WHAT TO TELL :

"Whenever code changes, Jenkins can automatically deploy the updated application using Docker. This is the essence of Continuous Deployment."